Stanford RoboSub

Electrical Onboarding 2026*

Stanford RoboSub Electrical Team

October 12, 2026

Contents

1 Setup **2**
- 1.1 Install Git and GitHub . . . 2
- 1.2 Clone and open the EE repository . . . 3
- 1.3 Install KiCad . . . 4
- 1.4 Install the Fabrication Toolkit . . . 5

2 The EE Repository **6**
- 2.1 Repository layout . . . 6
- 2.2 Where to start reading . . . 6
- 2.3 Shared parts library . . . 7
- 2.4 Working with the repository . . . 7

3 Learning KiCad **8**
- 3.1 Official learning resources . . . 8
- 3.2 Concepts to understand before your first board . . . 8
- 3.3 Using Claude with KiCad . . . 8

4 Starter Projects **10**
- 4.1 Hydrophone Analog Front-End . . . 11
- 4.2 Audio Firmware . . . 14
- 4.3 Challenge Project . . . 19

*Vibe coded: this guide was written with AI (Claude) and may have mistakes. If something is wrong, tell an EE lead.

1 Setup

By the end of this section you will have Git and the GitHub CLI installed, a copy of the `robosub-ee` repository on your laptop, and KiCad with the Fabrication Toolkit plugin ready to go. Each step has instructions for macOS, Windows and Linux; follow only the one for your machine.

- ☐ Create a GitHub account
- ☐ Install Git and the GitHub CLI (§1.1)
- ☐ Clone `robosub-ee` and open it (§1.2)
- ☐ Install KiCad (§1.3)
- ☐ Install the Fabrication Toolkit plugin (§1.4)

1.1 Install Git and GitHub

Git is the version-control tool that tracks every change to our schematics, layouts and firmware. **GitHub** is the website that hosts our repository. The **GitHub CLI** (`gh`) logs you in to GitHub from the terminal so you do not have to manage passwords or SSH keys by hand.

First, create a free account at `https://github.com/signup` if you do not already have one. The repository is public, so you can clone it without asking for access.

macOS

Open **Terminal** (press `Cmd+Space` and type "Terminal"). Install Homebrew, the macOS package manager, if you do not have it yet, by following the one-line command at `https://brew.sh`. Then:

```
brew install git gh
```

Note If you would rather not use Homebrew, running `git -version` prompts macOS to install Apple's Command Line Tools, which include Git. You can then download `gh` from `https://cli.github.com`.

Windows

Open **PowerShell** from the Start menu and run:

```
winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
```

Close and reopen PowerShell so it picks up the new programs. Alternatively, download the installers from `https://git-scm.com/download/win` and `https://cli.github.com`; the default options in the Git installer are fine. The Git installer also gives you **Git Bash**, a Linux-style terminal that the commands in this guide work in unchanged.

Linux

On Ubuntu or Debian, install Git from apt but install `gh` from GitHub's own package repository: the `gh` in Ubuntu's and Debian's repositories is either very old or a 2.45/2.46 build that no longer works.

```
sudo apt update
sudo apt install git wget -y
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -nv -O- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
    /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-
    archive-keyring.gpg] https://cli.github.com/packages stable main" | sudo tee /etc/
    apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install gh -y
```

On Fedora: `sudo dnf install git gh`. On Arch: `sudo pacman -S git github-cli`. If your distribution's `gh` is missing or very old, follow the official instructions at https://github.com/cli/cli/blob/trunk/docs/install_linux.md.

All platforms: configure Git and log in

Tell Git who you are (use the email attached to your GitHub account), then log in through the browser:

```
git config --global user.name "Your Name"
git config --global user.email "you@stanford.edu"
gh auth login
```

When `gh auth login` asks, choose **GitHub.com**, **HTTPS**, answer **Yes** to authenticating Git with your GitHub credentials, and **Login with a web browser**. Check that it worked:

```
git --version
gh auth status
```

Prefer a GUI? GitHub Desktop (macOS and Windows) does the same clone, commit and push steps with buttons. It is fine to use, but learn the terminal commands too: they are what everyone on the team will use when helping you.

1.2 Clone and open the EE repository

Our electrical work lives in one repository: https://github.com/ryos17/robosub-ee. "Cloning" downloads a full copy, including its history, to your laptop. Pick a folder for your projects and clone into it:

```
mkdir -p ~/repos
cd ~/repos
gh repo clone ryos17/robosub-ee
cd robosub-ee
```

(`git clone https://github.com/ryos17/robosub-ee.git` works too.) On Windows, run these in Git Bash, or in PowerShell with `mkdir $HOME\repos` in place of the first line.

Open the folder in your editor. We recommend Visual Studio Code:

```
code .
```

If `code` is not found on macOS, open VS Code, press `Cmd+Shift+P` and run **Shell Command: Install 'code' command in PATH**. You can also use **File** → **Open Folder**.

Staying up to date Other people push changes every week. Before you start working, pull the latest version with `git pull`. Section 2 walks through what is in the repository.

1.3 Install KiCad

KiCad is the free, open-source tool we use for schematics and PCB layout. Install the latest stable release from https://www.kicad.org/download/ (version 10 at the time of writing). Everyone on the team should be on the same major version: a board saved in a newer KiCad cannot be opened in an older one.

macOS

Download the `.dmg` from https://www.kicad.org/download/macos/, open it and drag the **KiCad** folder into **Applications**. Or, with Homebrew:

```
brew install --cask kicad
```

The first time you launch it, macOS may ask you to confirm that you want to open an app downloaded from the internet.

Windows

Download the installer from https://www.kicad.org/download/windows/ and run it with the default options (it includes the symbol, footprint and 3D model libraries, which is a large download). Or, from PowerShell:

```
winget install --id KiCad.KiCad -e --source winget
```

Linux

On Ubuntu, use the official KiCad PPA so you get the current release rather than the older one in Ubuntu's own repositories. The PPA name includes the version; check https://www.kicad.org/download/ubuntu/ for the current one and substitute it below:

```
sudo add-apt-repository ppa:kicad/kicad-10.0-releases
sudo apt update
sudo apt install kicad
```

On any other distribution, the Flatpak is the easiest option:

```
flatpak install flathub org.kicad.KiCad
```

See https://www.kicad.org/download/linux/ for distribution-specific packages.

All platforms: point KiCad at our shared parts library

Our custom symbols, footprints and 3D models live in `imported_parts/` in the repository. The footprints find their 3D models through a KiCad path variable, so set it once:

1. Open KiCad and go to **Preferences** → **Configure Paths...**
2. Add a variable named `ROBOSUB_3D_MODELS` whose path is the *absolute* path to `imported_parts/robosub_3d_models` inside your clone (for example `/Users/you/repos/robosub-ee/imported_parts/robosub_3d_models`).

3. Click **OK** and restart KiCad.

Boards that use the shared library, such as `gen3/PCBs/Micro_Breakout`, already list its symbols and footprints in their own project library tables, so the path variable is the only global setting you need. Section 2 covers this library in more detail.

1.4 Install the Fabrication Toolkit

The Fabrication Toolkit is a KiCad plugin that, in one click, exports everything a board house such as JLCPCB needs to make and assemble a board: the Gerber and drill files (zipped), the bill of materials (BOM) and the component placement (CPL) file. It installs the same way on every operating system:

1. Open KiCad and click **Plugin and Content Manager** in the main window.
2. Open the **Plugins** tab and search for **Fabrication Toolkit**.
3. Click **Install**, then **Apply Pending Changes** at the bottom.
4. Close and reopen the PCB Editor.

To use it, open a board in the PCB Editor and click the Fabrication Toolkit button in the top toolbar (or **Tools** → **External Plugins** → **Fabrication Toolkit**). It writes its output to a `production/` folder next to the board file.

On the Linux Flatpak build, if the plugin does not appear in the toolbar after installing, use **Tools** → **External Plugins** → **Refresh Plugins**.

2 The EE Repository

The `robosub-ee` repository holds everything the electrical subsystem of the Stanford RoboSub AUV produces: PCB designs (KiCad), firmware, Python tools and research. Its `README.md` is the source of truth for how it is organised. **Read it now**, either on GitHub or in the clone you made in Section 1.2; this section is a guided tour of the same layout. Most folders have their own `README.md` with more detail, and those are worth reading too.

Note If this section and the repository's README ever disagree, the README wins. Please tell an EE lead so this document can be fixed.

2.1 Repository layout

`gen1/`
The first hardware generation: PCBs only. **Deprecated**; that sub no longer exists. Useful only as history.

`gen2/`
The second, **current** generation: the vehicle that runs today. Contains the board designs (`PCBs/`), the Teensy 4.1 firmware (`teensy_firmware/`) for thrusters, sensors, the kill switch and telemetry, and the Electrosmith Daisy Seed audio firmware (`daisyseed_firmware/`) that reads the four hydrophones.

`gen3/`
The third generation, **in progress**: the Micro Breakout board, battery and power-management boards, and power distribution. `gen3/README.md` shows the proposed system diagram and the current to-do list.

`imported_parts/`
The shared KiCad library: symbols, footprints, 3D models, the import script and the JLC parts order log. New boards reference this library instead of keeping their own copies.

`misc/`
Older and experimental material.

`research/`
Research work, currently whisper-based inter-vehicle communication.

`platformio.ini`
PlatformIO configuration for building and flashing the Teensy firmware. Its `src_dir` points at one sketch folder; change it to choose which firmware gets flashed (the comments in the file explain how).

2.2 Where to start reading

1. `README.md` at the top of the repository.
2. `gen2/README.md`, for the system that actually runs, including its system diagram and known quirks.
3. `gen3/README.md`, for what is being designed right now.

4. `imported_parts/README.md`, before you add or use any custom part.
5. The README inside whichever folder your starter project (Section 4) points you to.

2.3 Shared parts library

`imported_parts/README.md` describes the workflow; in short, when you add a custom part:

1. Add or merge its symbol into `robosub_symbols/robosub_symbols.kicad_sym`.
2. Add its footprint to `robosub_footprints.pretty/`.
3. Add its 3D model to `robosub_3d_models/` and reference it from the footprint as `${ROBOSUB_3D_MODELS}/...` (you set this variable up in Section 1.3).
4. Reopen or refresh the libraries in KiCad.

`jlc_to_robosub.py` automates importing JLC/EasyEDA parts into this structure, and `robosub_jlc_parts_log.csv` records what has already been imported so parts are not added twice.

2.4 Working with the repository

- Run `git pull` before you start working.
- Do your work on a branch rather than directly on `main`:

  ```
  git switch -c yourname/what-you-are-doing
  ```

- Commit small, related changes with a message that says what changed and why, then `git push` and open a pull request on GitHub for review.
- KiCad keeps a lock file while a design is open. Close KiCad before pulling changes to a board someone else may have edited, and talk to whoever owns a board before editing it: two people's layout changes to the same board cannot be merged automatically.

3 Learning KiCad

Eli runs a hands-on KiCad tutorial at one of our meetings, and that is the main way to learn it. Use this section if you missed that meeting or want a refresher afterwards.

3.1 Official learning resources

KiCad maintains a list of books, written tutorials and video tutorials at

`https://www.kicad.org/help/learning-resources/`

A good path through it:

1. **Getting Started in KiCad**, in the official documentation (`https://docs.kicad.org`). It takes you through a complete small project: drawing a schematic, assigning footprints, laying out a board and generating fabrication files. Do this one end to end.
2. Pick one of the video series from the learning-resources page and follow it while building a board of your own. Make sure it is for KiCad 8 or newer; older tutorials still teach the right ideas, but menus have moved.
3. Keep the reference manuals for the **Schematic Editor** and **PCB Editor** (also on `https://docs.kicad.org`) open as you work, and look things up as they come up.

3.2 Concepts to understand before your first board

- **Symbols vs. footprints vs. 3D models**: the schematic symbol is the logical part, the footprint is its copper pads on the board, and the 3D model is only for visualisation and mechanical checks.
- **Libraries and library tables**: where KiCad looks for symbols and footprints, and why we keep our own in `imported_parts/` (Section 2).
- **ERC and DRC**: the Electrical Rules Checker (schematic) and the Design Rules Checker (board). ERC is good practice, but you can ask Claude to check your schematic instead. **DRC is mandatory**: fix every DRC error before a board is ordered.
- **Net classes and design rules**: trace width, clearance and via sizes, which must match what the board house can manufacture.
- **Fabrication outputs**: Gerbers, drill files, BOM and placement files. The Fabrication Toolkit (Section 1.4) produces all of them for JLCPCB.

3.3 Using Claude with KiCad

Claude is very good at KiCad, and you are encouraged to use it as a tutor and a reviewer while you learn. Things it does well:

- Explaining what a KiCad feature, error or DRC violation means and how to fix it.
- Walking you through a workflow step by step ("how do I add a differential pair", "how do I make a custom footprint from this datasheet").

- Reviewing a circuit: paste in or describe your schematic and ask what is missing, such as decoupling capacitors, pull-ups, or protection on connectors.
- Reading KiCad's files directly. `.kicad_sch` and `.kicad_pcb` files are plain text, so tools like Claude Code can open a project in the repository and answer questions about it.

Check its work Treat Claude's answers the way you would treat a knowledgeable teammate's: usually right, occasionally confidently wrong. Always confirm part numbers, pinouts and footprint dimensions against the manufacturer's datasheet, and have an EE lead review every board before it is ordered.

4 Starter Projects

Each new member picks one starter project. They are small enough to finish in a few weeks, and each one exercises a real part of the sub.

Project	Track	Summary
4.1 Hydrophone Analog Front-End	Hardware	Filtering, gain and input clamps for a hydrophone signal.
4.2 Audio Firmware	Firmware and Signal Processing	Set up a Daisy Seed and write audio firmware for it.
4.3 Challenge Project	All of the above	Four-channel, 200 kHz-target hydrophone front end and recorder.

4.1 Hydrophone Analog Front-End

Overview

Our hydrophones are large piezo disks glued to acrylic, each on about 2 ft of cable, and right now they plug straight into the Daisy Seed. Their signal is tiny and full of noise, so in this project you draw, in KiCad, a small circuit that cleans it up and makes it louder before it reaches the Daisy Seed.

What you need

- KiCad, set up as in Setup (Section 1).
- Learning KiCad (Section 3) and Eli's tutorial.

How it works

The signal passes through four blocks, in this order:

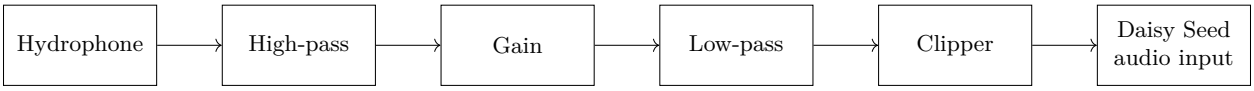


Figure 1: One hydrophone channel.

High-pass
Removes sounds *below* the band we care about, such as rumble from the thrusters and the water moving past the sub.

Gain
Makes the signal 10 times bigger, so the Seed can hear it clearly.

Low-pass
Removes sounds *above* the band we care about, mostly hiss.

Clipper
Four diodes that stop an unexpectedly loud sound from going past what the Seed's input can take.

The circuit runs from a single 3.3 V supply, so the signal swings around 1.65 V (half of 3.3 V) instead of around 0 V. That middle voltage is called V_{REF} and comes from two equal resistors (Figure 3).

Two presets. The high-pass and low-pass decide which band of sound gets through. Pick one:

Preset	Keeps	For
Voice	100 Hz – 8 kHz	speech
Pinger	10 kHz – 50 kHz	the 25–40 kHz competition pingers

Only two capacitors change between the two (C_1 and C_2 in the parts list).

The circuit

Figure 2 is what you will draw, and Figure 3 makes V_{REF}.

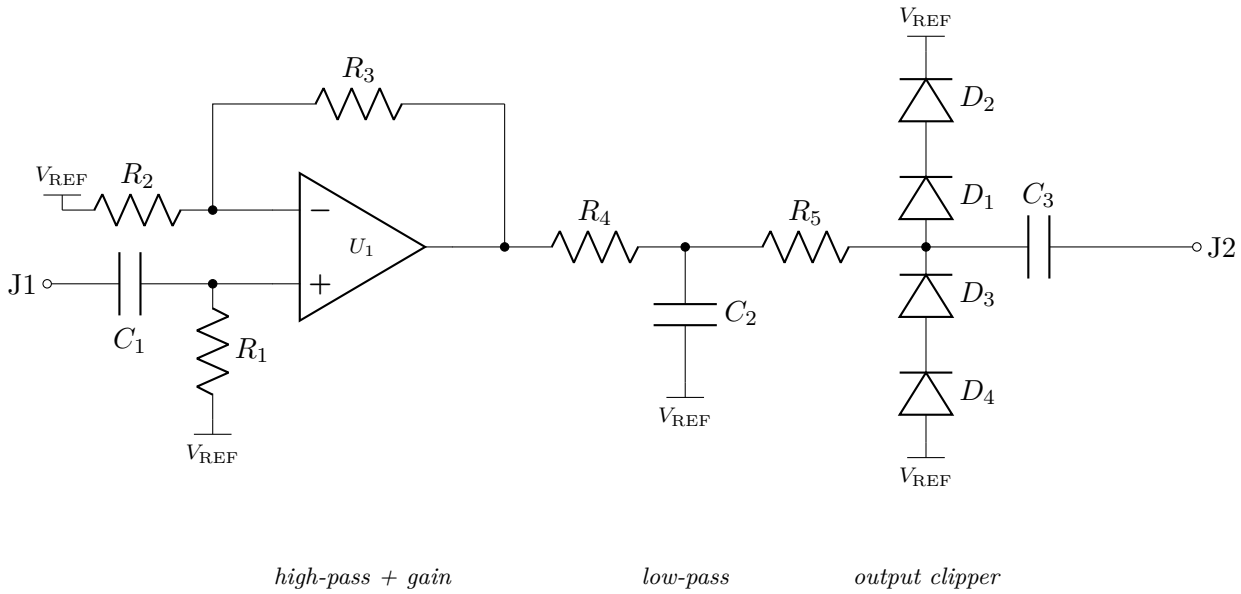


Figure 2: One channel. J1 is the hydrophone, J2 goes to a Daisy Seed audio input (on the breakout board from the Audio Firmware project, Section 4.2). The op-amp's power pins are not drawn.

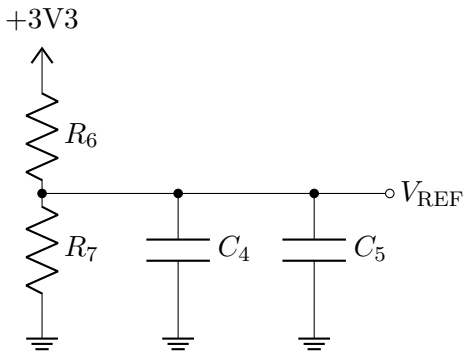


Figure 3: V_{REF}: two equal resistors split 3.3 V in half (1.65 V); C_4 and C_5 keep it steady.

Parts list.

Part	Voice	Pinger	What it does
C_1	22 nF, C0G	220 pF, C0G	high-pass
R_1	71.5 kΩ		high-pass
R_2	1 kΩ		gain
R_3	9.09 kΩ		gain (with R_2, sets ×10)
R_4	909 Ω		low-pass
C_2	22 nF, C0G	3.6 nF, C0G	low-pass
R_5	100 Ω		output resistor
C_3	1 µF		blocks the 1.65 V offset from the Seed
D_1–D_4	1N4148W		clipper
R_6, R_7	10 kΩ		make V_{REF}
C_4, C_5	10 µF, 100 nF		keep V_{REF} steady
U_1	OPA365 (op-amp)		amplifier

Resistors are 1%. All resistors and capacitors use **0805** footprints. Also add a 100 nF and a 10 µF capacitor from 3.3 V to ground next to the op-amp's power pins.

Done when the schematic is drawn and every part has a value and a footprint. Checking it with ERC is good practice, or ask Claude to review it.

Stretch goals

- Make the channel a hierarchical sheet and place it four times, one per hydrophone.
- Simulate it in KiCad (ngspice) and check that the band you picked gets through.

For reference: the formulas These are how the part values were chosen; you do not need them to finish.

$$f_{HP} = \frac{1}{2\pi R_1 C_1}, \qquad G = 1 + \frac{R_3}{R_2}, \qquad f_{LP} = \frac{1}{2\pi R_4 C_2}, \qquad V_{clip} \approx 2 \times 0.65\,\text{V} \approx 1.3\,\text{V}.$$

The clip level is two diode drops on each side of V_{REF}, safely under the Seed's ±1.8 V input limit.

Resources

- KiCad documentation: https://docs.kicad.org.
- Daisy Seed datasheet, for the audio input limits: https://docs.daisy.audio/hardware/Seed/.

4.2 Audio Firmware

Track: Firmware and Signal Processing **Difficulty:** Medium
Time: ∼2–3 weeks

Overview

The sub listens for an acoustic pinger with four hydrophones, read by two Electrosmith Daisy Seed boards. In this project you design the simple breakout board those two Seeds plug into, then write audio firmware for a Seed: first passing audio through, then filtering it, then finding frequencies with an FFT, and optionally estimating the direction a sound comes from. If you need to, you can skip the PCB design and wire everything on a breadboard instead.

Prerequisites

- Finished Setup (Section 1) and Learning KiCad (Section 3).
- Some C or C++. No signal-processing background is needed; the equations you need are below.

What you need

- A Daisy Seed and a micro-USB cable.
- A breadboard and jumper wires.
- Something to listen to: a hydrophone, any transducer (a piezo disc works), the headphone output of a laptop or phone, or a signal generator.

Part 1: Design the breakout board

Two Daisy Seeds (Seed 1.1, Rev5), a master and a slave, read four hydrophones at 96 kHz, two per board. The master reads hydrophones 0 and 1 and talks to the Orin over USB serial. The slave reads hydrophones 2 and 3 and reports their levels to the master as analog voltages (slave DAC to master ADC). There are no preamps or extra parts, only direct connections, and each Seed plugs into female headers so it can be removed.

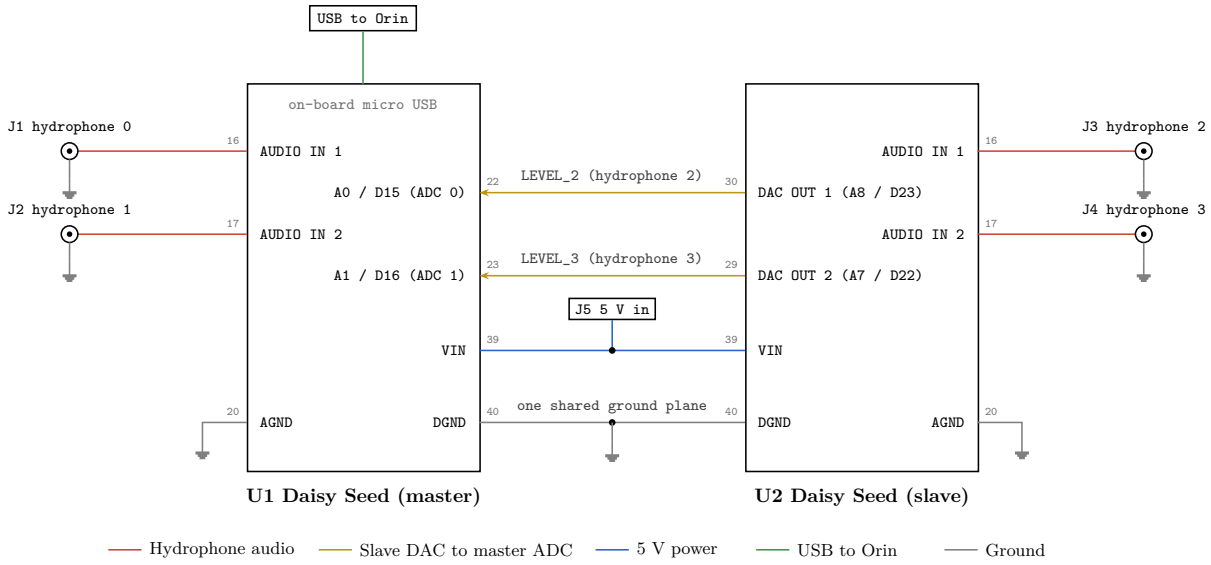


Figure 4: Daisy Seed breakout board. Numbers are the Seed's physical header pins.

Connections (physical header pin numbers)

From	To	Carries
Hydrophone 0 signal	Master pin 16 (Audio In 1)	audio
Hydrophone 1 signal	Master pin 17 (Audio In 2)	audio
Hydrophone 2 signal	Slave pin 16 (Audio In 1)	audio
Hydrophone 3 signal	Slave pin 17 (Audio In 2)	audio
All hydrophone shields	Ground	
Slave pin 30 (DAC Out 1)	Master pin 22 (A0, ADC 0)	level of hydrophone 2, 0–3.3 V
Slave pin 29 (DAC Out 2)	Master pin 23 (A1, ADC 1)	level of hydrophone 3, 0–3.3 V
J5 5 V input	Pin 39 (VIN), both boards	power for both Seeds
Pins 20 (AGND), 40 (DGND), both boards	One shared ground plane	
Master micro-USB	Orin	USB serial, no PCB traces

Design notes

- **Shared ground.** The master's ADC measures the slave's DAC voltage relative to its own ground, so both boards need the same 0 V reference. Join all four ground pins on one solid, unbroken plane under both Seeds. (The Seed's datasheet also requires AGND to be connected to DGND.)
- **Power.** One 5 V source (J5) feeds the VIN pin of both Seeds (VIN accepts 4–17 V). You cannot power the slave from the master's VIN pin: VIN is an input only, and on a USB-powered Seed it carries no voltage. The master keeps its micro-USB for data, and the Seed is safe to power from VIN and USB at the same time.

- **Ground loops.** If each Seed had its own USB cable to the Orin, their grounds would also connect through the Orin, making a ground loop that can add noise to the hydrophone signals. The same happens if J5 is a second USB cable from the Orin, so take 5 V from somewhere else on the sub.
- **Signal level.** The Seed's audio inputs expect line level (about $1\,V_{rms}$, AC-coupled, $30\,k\Omega$). A hydrophone puts out millivolts, so with direct connections the signal is small. A preamp fixes this; that is what the Hydrophone Frontend project (Section 4.1) builds.
- **Layout.** Keep the hydrophone traces short and away from the DAC lines.

Done when the layout passes DRC (mandatory) and the board has been reviewed. Run ERC on the schematic too, or ask Claude to check it.

Part 2: Write the firmware

Set up. Follow Electrosmith's official guide, *C++ Development Environment*, which sets up VS Code (not Max/MSP or Arduino). There is also a video walkthrough of the same steps. In short:

1. Install the toolchain for your OS: macOS, Windows (use Git Bash as the VS Code terminal), or Linux.
2. Clone the examples, which include libDaisy:
   ```
   git clone --recurse-submodules https://github.com/electro-smith/DaisyExamples
   ```
3. Open the folder in VS Code and run the task `build_all` once.
4. Open `seed/Blink`, put the Seed in bootloader mode (hold BOOT, tap RESET, release BOOT) and run the task `build_and_program_dfu`. The onboard LED should blink.

If flashing gives you trouble, the Daisy Web Programmer flashes a `.bin` from Chrome with no toolchain.

Milestone 1 (required): pass-through and gain. Make a new folder next to `seed/Blink` (for example `seed/Gain`) with these two files. The program copies both inputs to both outputs, scaled by `GAIN`. Wire an audio source into Audio In 1 and 2 and headphones or a speaker to Audio Out 1 and 2, then build and flash with `make && make program-dfu` (with the Seed in bootloader mode). Change `GAIN` and listen to the difference.

`main.cpp`

```
#include "daisy_seed.h"

using namespace daisy;

DaisySeed hw;                       // the Daisy Seed board itself

constexpr float GAIN = 0.5f;        // 0.5 = half volume (about -6 dB)

// Called by the audio hardware every time a new block of samples is ready.
// in[0][i] / in[1][i] are the left / right inputs, out[...] the outputs.
void AudioCallback(AudioHandle::InputBuffer  in,
```

```
                    AudioHandle::OutputBuffer out,
                    size_t                    size)
{
    for(size_t i = 0; i < size; i++)
    {
        out[0][i] = in[0][i] * GAIN; // left  in -> left  out
        out[1][i] = in[1][i] * GAIN; // right in -> right out
    }
}

int main(void)
{
    hw.Init();                      // set up clocks, memory and pins
    hw.SetAudioBlockSize(4);        // 4 samples per callback = low latency
    hw.SetAudioSampleRate(SaiHandle::Config::SampleRate::SAI_96KHZ);
    hw.StartAudio(AudioCallback);  // from now on AudioCallback runs on its own

    while(1) {}                     // nothing else to do; audio runs in the background
}
```

Makefile

```
# Project name (the output will be build/gain.bin)
TARGET = gain

# Source files
CPP_SOURCES = main.cpp

# Path to libDaisy (relative to this folder)
LIBDAISY_DIR = ../../libDaisy

# Pull in libDaisy's build rules
SYSTEM_FILES_DIR = $(LIBDAISY_DIR)/core
include $(SYSTEM_FILES_DIR)/Makefile
```

At 96 kHz a block of 4 samples means the callback runs about every 42 µs. That is fine for a gain stage; for the heavier processing below, use a bigger block such as 48. To print numbers to your laptop, see the serial printing tutorial (`hw.StartLog()` and `hw.PrintLine()`).

Milestone 2 (optional): band-pass filter. A pinger sits in a narrow frequency band, so the first useful step is to keep that band and throw away everything else. A biquad is a second-order filter; each output sample depends on the last two inputs and the last two outputs:

$$y[n] = b_0\,x[n] + b_1\,x[n-1] + b_2\,x[n-2] - a_1\,y[n-1] - a_2\,y[n-2].$$

For a band-pass filter with centre frequency f_0 and quality factor Q at sample rate f_s (from the widely used *Audio EQ Cookbook*):

$$\omega_0 = \frac{2\pi f_0}{f_s}, \qquad \alpha = \frac{\sin\omega_0}{2Q},$$

$$b_0 = \frac{\alpha}{1+\alpha}, \quad b_1 = 0, \quad b_2 = -\frac{\alpha}{1+\alpha}, \quad a_1 = \frac{-2\cos\omega_0}{1+\alpha}, \quad a_2 = \frac{1-\alpha}{1+\alpha}.$$

The bandwidth is roughly f_0/Q. Compute the coefficients once at start-up, keep $x[n-1], x[n-2], y[n-1], y[n-2]$ between calls to the audio callback, and run the equation on every sample. Test it by sweeping a tone through the band and printing the output level.

Milestone 3 (optional): FFT. An FFT turns a block of N samples into N frequency bins, so you can see which frequencies are present. It computes the discrete Fourier transform

$$X[k] = \sum_{n=0}^{N-1} w[n]\, x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, \ldots, N-1,$$

where $w[n] = \frac{1}{2}(1 - \cos\frac{2\pi n}{N-1})$ is a Hann window that reduces leakage between bins. Bin k corresponds to frequency $f_k = kf_s/N$, so the resolution is f_s/N (at $f_s = 96\,\text{kHz}$ and $N = 1024$, about 94 Hz). For real input only bins 0 to $N/2$ are meaningful. Collect N samples, window them, take the FFT, and report the frequency of the largest $|X[k]|$.

Milestone 4 (optional, advanced): direction finding. Two hydrophones a distance d apart hear the same ping at slightly different times. If the sound arrives at angle θ from broadside, the extra path to the far hydrophone is $d\sin\theta$, so the delay is

$$\Delta t = \frac{d\sin\theta}{c}, \qquad \theta = \arcsin\left(\frac{c\,\Delta t}{d}\right),$$

with $c \approx 1500\,\text{m/s}$ in water. Estimate Δt as the lag τ that maximises the cross-correlation of the two channels,

$$R[\tau] = \sum_n x_1[n]\, x_2[n+\tau], \qquad \Delta t = \tau/f_s.$$

One sample at 96 kHz is about 10.4 μs, or 1.6 cm of path, which limits the angle resolution when d is small. *Delay-and-sum beamforming* is the same idea turned around: delay each channel by the amount expected for a given direction and add them; the direction with the loudest sum is where the sound came from.

Done when Milestone 1 runs on a Seed with a breadboard setup. Milestones 2–4 are optional.

Resources

- `gen2/daisyseed_firmware/` in `robosub-ee`: the firmware running on the sub today, and its README on building libDaisy and flashing.
- Daisy Seed hardware page, with the pinout and datasheet.
- libDaisy and DaisyExamples on GitHub.
- Daisy on YouTube.

4.3 Challenge Project

my words

As you have heard there is the easier project for the hydrophone front end but I thought that there should be something better so that we are good for both audio capture and higher frequency acoustic capture, potentially something like 200 kHz. We want input protection since piezo spikes can be pretty high voltage, some hi-Z input preamp with low noise, configurable gain and filter cutoff. I see my bot decided we should have some differential input. I think it has something to do with driving the ADC. Then some more input protection before the ADC. I asked my bot to pick a chip that can get us that potential 4-channel 200 kHz.

So yeah, basically a higher quality analog front end for the piezos. I know that it probably doesn't matter for this front end to be this nice but often capturing quality at ultrasonic frequencies means having a better design/board layout. This is meant mainly to be a good onboarding activity that is easy to explain that would be useful to the sub. We have a bunch of other projects that are more open-ended and not very well scoped yet (I've been chatting with Felicie about some details, like weight we can add to the sub for potential hydro speakers and such). Later on when we sort out teams and stuff we can talk about the open-ended projects that we need.

Oh, it also dawned on me that we want to use the piezos as ultrasonic hydro speakers. For this I think you should pick an external DAC for the Daisy Seed that has as high bandwidth as we can get (maybe outputting 200 kHz?) then we want some amplifier that can drive the piezos, but we prob want some output protection to ensure that we don't crack the piezos bc they really can't be that loud. For Felicie's onboarding she is putting together properly impedance-matched piezos for the water that we can for sure broadcast much more power into the water than the current acrylic plates (which are hi-Z in water, sorta fine for capturing but heinous for broadcasting). Since we'd be using the same hydrophone for receiving and transmitting with whatever circuit board this is we need some sort of switch to route the hydrophone to input vs output.

AI-generated engineering notes below. Probably most useful to give to your AI to chew on. The rest of this challenge section, including its schematics and calculations, is AI-generated and needs engineering review. Targets are requirements to test, not claims of measured performance.

Track: Hardware, firmware and signal processing **Difficulty:** Challenge
Time: Scoped prototype

Overview and status

Build a four-channel piezo front end and demonstrate synchronized recording. The baseline below fixes the capture and storage targets so the onboarding work has a clear finish line. Component values are a starting design for simulation and bench review; this is not a fabrication-ready schematic. The simpler projects (Sections 4.1 and 4.2) provide the background. The calculations and source notes are in `project-materials/hydrophone-analog-frontend/`.

Starting point

Per Eli, the hydrophones are large piezo disks superglued to acrylic with an airtight seal. The back of each disk is air, and the acrylic front faces the water. The hydrophones currently connect directly to the Daisy Seed audio inputs. There are four hydrophones, each with a cable of approximately

2 ft (0.61 m). The piezo model, its capacitance, the cable type, the Seed revision and any existing circuitry remain unverified, and the project migration is still in progress.

Requested behaviour: prototype acceptance targets

These are the proposed baseline requirements for this challenge. Changing one requires updating the filter, acquisition and storage budgets together.

Four synchronized inputs.
Four hydrophones, approximately 2 ft cables; one ADC with simultaneous channels on a shared clock. Preserve channel order and sample indices. Measure residual interchannel gain/phase error; do not silently mix independently clocked recordings.

Capture.
ADS127L14 wideband filter, max-speed mode, OSR 32, 32.768 MHz internal ADC clock, 512,000 samples/s *per channel*, 24-bit signed samples. The 24-bit word size is not a claim of 24 effective bits.

Two electrical passbands.
Audio: 20 Hz–20 kHz. Wideband: 20 Hz–200 kHz. Target magnitude within ±1.5 dB of the 1 kHz gain in either mode, measured through the electronics with a representative piezo/cable source model. The assembled underwater hydrophone must be characterized separately. Daisy selects the filter bank through GPIO expanders and signal relays; characterize contact parasitics and switch only while acquisition is blanked.

Gain and filtering.
Daisy selects nominal gain 1, 2 or 11 with fixed resistors and normally-open relay contacts; feedback stays connected. Keep the first high-impedance stage at unity gain. Use a nominal 5 Hz high-pass after that buffer and two second-order low-pass sections (values below). Hold gain and filter settings fixed during a capture; record them in the file metadata.

Local recording.
Write all four channels at the full rate in both filter modes. Packed 24-bit storage is 6.144 MB/s = 22.1184 GB/hour. Use native **four-bit SDMMC** for microSD; QSPI is a different protocol and is not needed for the card. Start with a SanDisk MAX ENDURANCE 256 GB U3/V30 card, with exact-card benchmarking required.

Storage margin.
Allocate a 16 MiB ring in the Seed's SDRAM, plus DMA working buffers. If the ring uses 32-bit words it holds 2.048 s of samples; packed storage holds 2.731 s. Qualify uninterrupted 60-minute recording, zero dropped/repeated frames, and no buffer overrun. Target at least 12.288 MB/s sustained packed writes on the actual host/card (2× payload). Exercise a 0.5 s writer stall and log real maximum stalls.

File format.
Interleave CH0, CH1, CH2, CH3 as signed 24-bit little-endian samples, 12 bytes per sample frame. Use files with at most 1 GiB of payload, rounded down to a multiple of 12 bytes, plus a documented header. Include sample rate, channel map, gain/filter settings, first sample index, calibration identifier and overrun counters. Rotate files without stopping acquisition; verify SDXC/filesystem support on the chosen firmware.

Host feasibility gate.

First prove the ADC's actual frame-sync data port can be captured continuously and written to SD. Daisy's documented 192 ksample/s SAI support does not certify this 512 ksample/s path. If it fails, change the acquisition interface/controller; do not quietly reduce the 200 kHz goal. No layout release until this gate passes.

Initial engineering direction (proposed)

Analog signal path. Figure 5 shows the current analog signal path, which is repeated for all four hydrophones.

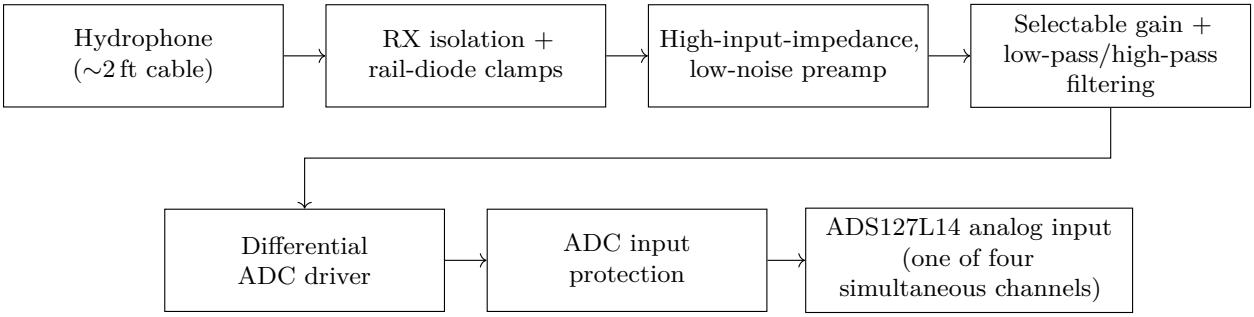


Figure 5: Proposed analog signal path, repeated for each of the four hydrophones.

The circuits: one channel, repeated four times

These are component-level design sketches for KiCad and simulation, using the same schematic notation as the easier project. Net names connect between figures. Values marked "select" require the stated fault/noise calculations; they are not unspecified wires or a promise of protection. Power pins and package pin numbers are omitted from the signal drawings; add them from the chosen package datasheets before ERC. All grounds here are the same analog reference, with continuous return paths in the PCB.

1. Rail-steering diodes and high-impedance buffer. Use two low-leakage diode junctions per hydrophone: one from the protected node to +5VA, one from -5VA to that node. One BAV199 dual package provides both junctions; connect its series midpoint to the signal. They are reverse biased throughout the provisional ±2 V normal input range. No extra protection supplies or fault-protection IC are needed.

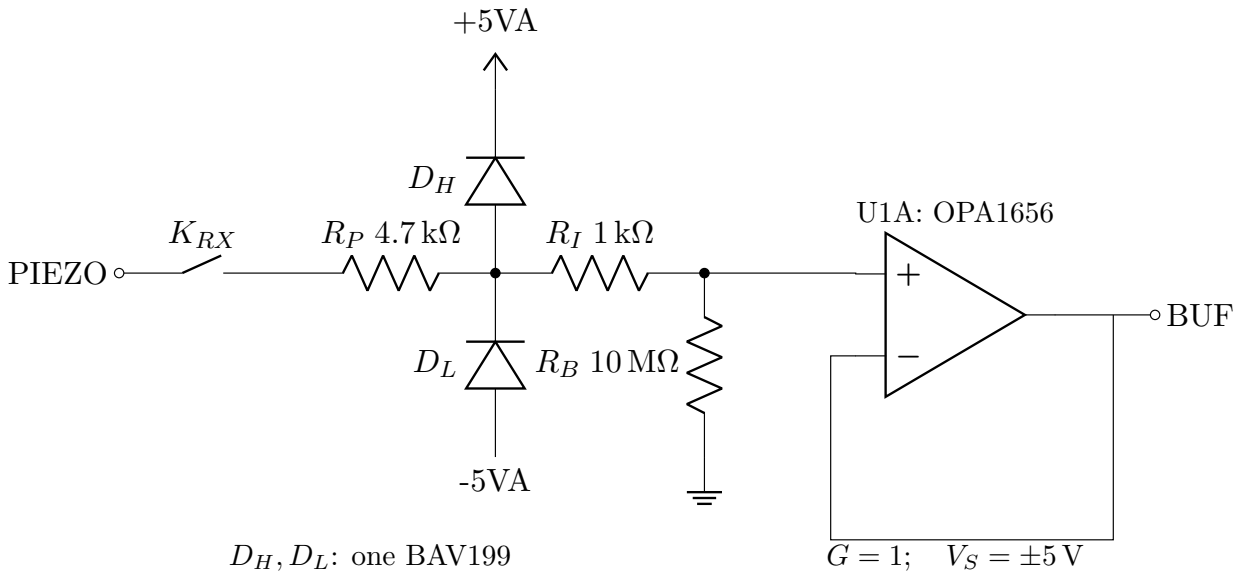


Figure 6: Two opposite-rail steering diodes per sensor; four dual packages for four sensors. Contacts shown unpowered.

R_P limits external clamp current; R_I limits current into the buffer's internal input clamps when the silicon steering diode's forward drop exceeds the op amp's allowed rail excursion. TI explicitly permits current-limited input overvoltage, with a 10 mA absolute maximum; target below 2 mA at the buffer pin and verify transient behavior. Do not assume a silicon diode holds the pin within 0.5 V of its rail. At a 50 V connector pulse and approximately 6 V clamp level, R_P carries about 9.4 mA and dissipates 0.41 W while the pulse lasts. Select a pulse-rated resistor by measured pulse energy and voltage; 50 V is a test example, not a certified board limit.

Both rails need a defined path for injected charge: start with 47 μF local bulk capacitance per rail, in addition to amplifier bypassing. Four 10 nF piezos each falling from 50 to 6 V inject roughly 1.76 μC total, or 37 mV into 47 μF before load/regulator effects. Size this from measured sensor capacitance and simultaneous faults; check rail recovery, regulator reverse-current tolerance and capacitor tolerance. Sustained external voltage is outside this preliminary piezo-pulse envelope. RX stays open until analog power is good and opens for TX; test power loss during relay release as well.

The 5.7 kΩ total series resistance contributes about 9.7 nV/$\sqrt{\text{Hz}}$ at 300 K before filtering, exceeding the op amp's own broadband voltage noise. This is a deliberate conservative starting value; reduce it only after measuring the required fault envelope. Include diode, relay and buffer capacitance, hot leakage, board leakage and the sensor/cable impedance in the noise/loading model. Compute leakage-induced offset across R_B and measure linearity at 20 and 200 kHz. Rail clamps protect against faults; they do not prevent amplifier saturation or guarantee distortion-free recording of arbitrary amplitudes. Mark overload and recovery samples invalid.

2. Daisy-selected gain with permanent feedback. $C_H = 330$ nF and $R_H = 100$ kΩ retain the 4.82 Hz high-pass. The 10 kΩ feedback resistor is always connected. Daisy closes one of two normally-open signal-relay contacts to add a gain-setting resistor to ground: 10 kΩ for gain 2 or 1 kΩ for gain 11. Both open gives unity gain, so power-up or a broken control connection cannot open the feedback loop. Both closed gives gain 12 and is an invalid command.

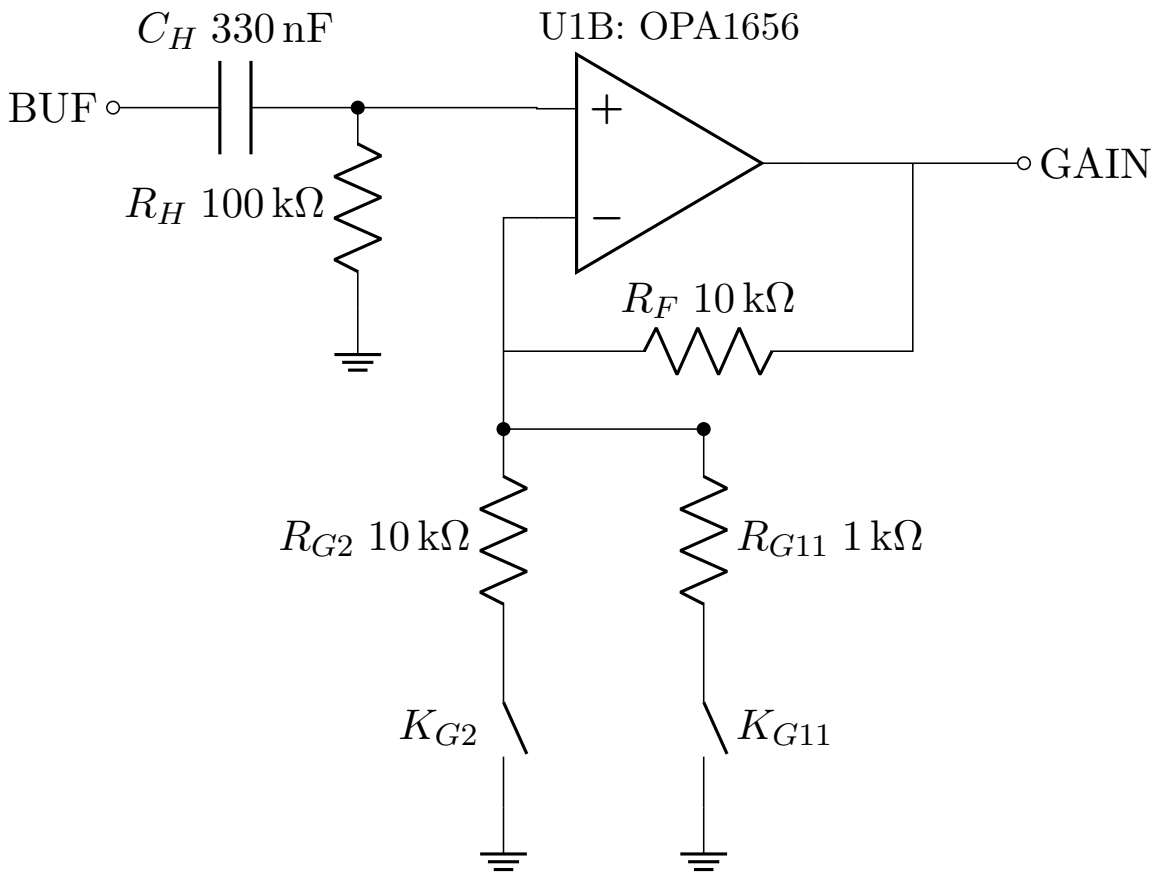


Figure 7: Relay-selectable gain. Contacts shown de-energized: gain 1. Coil drivers are on the control sheet.

For a 2.4 V differential ADC headroom target, the input limit is $\min(2, 2.4/G)$ V peak: 2.0, 1.2 and 0.218 V for gains 1, 2 and 11. The 2.4 V stage limit leaves margin below the OPA1656 positive common-mode limit of +2.75 V on +5 V supply. These are nominal limits, before offsets, filter peaking and tolerances. Select gain during a preflight measurement; freeze it during a recording. Automatic gain changes, if later added, occur between blocks, with gain metadata and settling gaps. AGC cannot catch the first fast spike and is not a replacement for the passive input protection.

3. Selectable low-pass: two identical sections in cascade. Build the circuit below twice per channel: GAIN → LP1 → LP2. Both op amps are OPA1656 sections on ±5 V rails. Use a complete matched capacitor bank for each mode (two capacitors per section, four per channel), selected by Daisy-driven, nonlatching DPDT signal relays. Two relays per channel switch the four capacitor legs together: de-energized is wideband, energized is audio. Each unused capacitor has one end disconnected. Use low-capacitance signal contacts, not power relays; include contact resistance, open-contact capacitance and coil coupling in the model.

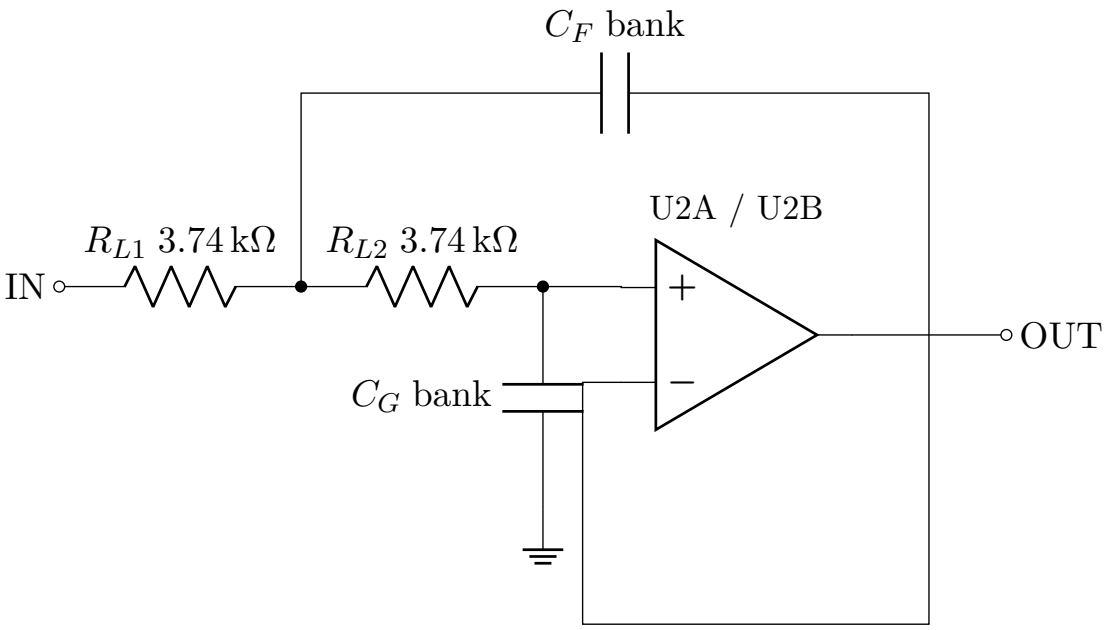


Figure 8: Unity-gain Sallen–Key section with real feedback and shunt capacitors. Repeat twice.

Bank	C_F	C_G	f_0 per section	Q per section
Audio	1.64 nF	820 pF	36.70 kHz	0.707
Wideband	160 pF	82 pF	371.5 kHz	0.698

Use 0.1% matched resistors and 1% C0G capacitors (parallel parts are fine). For equal resistors R,

$$f_0 = \frac{1}{2\pi R\sqrt{C_F C_G}}, \qquad Q = \frac{1}{2}\sqrt{\frac{C_F}{C_G}}.$$

The complete low-pass is two cascaded sections, not a fourth-order Butterworth response. Nominal attenuation is about 0.73 dB at 20 kHz in audio mode and 0.82 dB at 200 kHz in wideband mode; the exact values must be checked against the acceptance requirements. The accompanying `calculate_filter_response.py` checks nominal response and all 256 low-pass resistor/capacitor tolerance corners, including the high-pass and ADC reservoir pole. It is an ideal linear model, not a stability/noise simulation. These are analog shaping filters; the ADC's wideband digital filter provides the sharp transition near Nyquist. Check aliasing around modulator image frequencies as well.

4. Single-ended to differential ADC driver. The piezo remains single-ended through LP2. U3 makes a balanced pair locally at the ADC; it does not cancel noise already picked up by the cable. $V_{\mathrm{AINP}} - V_{\mathrm{AINN}} \approx V_{\mathrm{LP2}}$ at low frequency for equal resistor ratios; their average follows the ADC VCM output (nominally 2.5 V with the proposed 5 V analog supply). VCM is not the 4.096 V conversion reference.

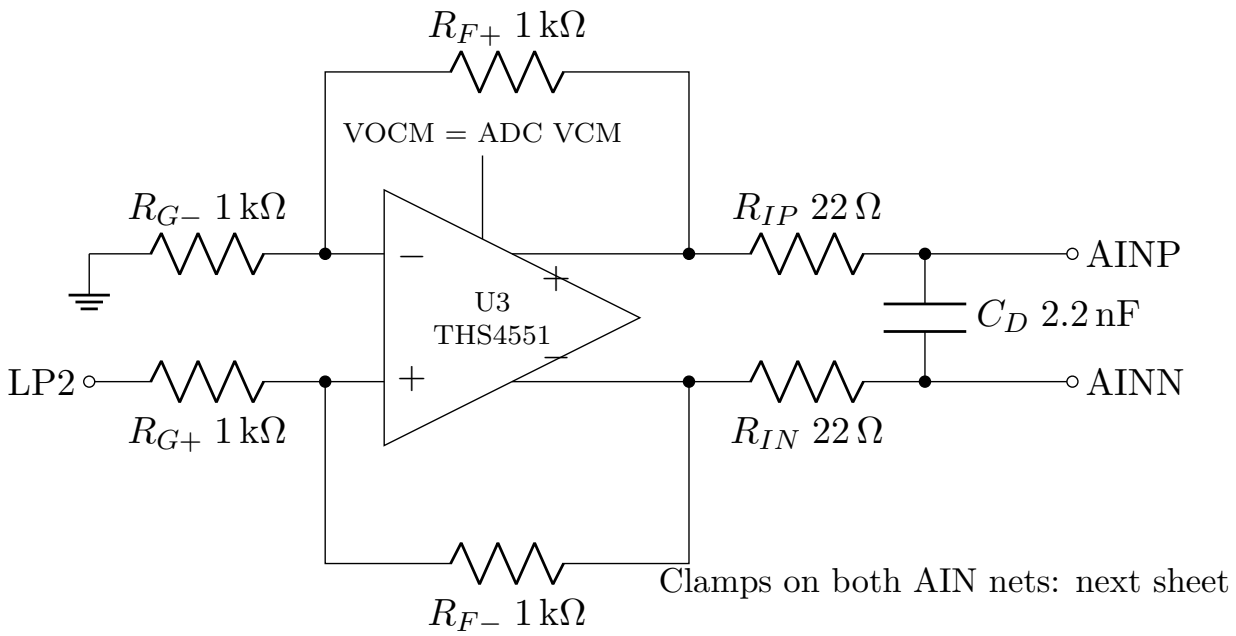


Figure 9: THS4551 on +5 V/AGND; matched 0.1% gain resistors. ADC on the same analog supply.

The 22 Ω/2.2 nF differential reservoir has a nominal pole at $1/[2\pi(22+22)\,\Omega \cdot 2.2\,\text{nF}] = 1.64\,\text{MHz}$. It adds about 0.064 dB loss at 200 kHz. This sketch uses resistive FDA feedback; it is not a copy of TI's complete active anti-alias filter. Simulate driver stability, ADC loading and common-mode settling using the actual models. A 4.096 V reference in the ADC's 1× standard range corresponds to nominal ±4.096 V differential full scale; keep ordinary peaks below ±2.4 V differential for headroom.

5. ADC clamps and support circuitry. Add this clamp network at *each* AINP/AINN node, after its 22 Ω resistor. Diode choice remains a design exercise: ordinary silicon diodes are not automatically adequate for the ADC's rail limits. The receiving rail must absorb clamp current, including during power sequencing.

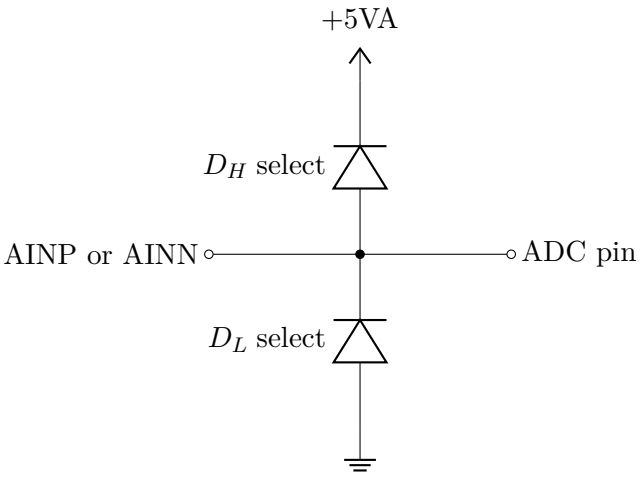


Figure 10: Upper diode: pin to +5VA. Lower diode: AGND to pin. Not validated for powered-off faults.

Check pin voltage stays within the ADC's absolute limits and continuous pin current within its rating (datasheet: rails ±0.3 V and 10 mA). Use a tighter operating target; include diode forward voltage at fault current, rail rise, leakage, capacitance and the driver fault current. If 22 Ω is insufficient, redesign the clamp/driver network and recheck settling and noise. Do not call this a proven protection circuit until the fault envelope and parts are specified.

Provide low-noise +5VA/-5VA for the OPA1656 stages, share +5VA with U3/ADC, and 1.8 V ADC I/O with direction-controlled 1.8/3.3 V translation. Use a precision 4.096 V reference and its recommended buffer/decoupling, with ADC REFN at AGND. Add 100 nF at every amplifier supply pin and local bulk capacitors; implement ADC CAPA/CAPD and reference bypassing exactly as specified in its datasheet. These power/reference circuits and the acquisition receiver need complete KiCad sheets before layout; their parts are not selected.

Parts count and acceptance. These schematics use four OPA1656 amplifier sections per channel, hence **eight dual OPA1656 packages for four channels**, plus four THS4551s and one ADS127L14. The earlier two-package estimate counted only one preamp per channel and omitted the gain/filter stages. The initial passive values must pass the response calculation, tolerance simulation, input-noise budget and bench tests before being frozen.

Protection sources. BAV199 datasheet provides leakage, capacitance and diode pulse limits; OPA1656 datasheet, input protection section defines current-limited overload operation. Sensor and ADC clamps have different leakage, voltage and current requirements; do not reuse the sensor diode choice at the ADC without checking its tighter limits.

Daisy configuration and transmit/receive control

Configuration is a required part of this PCB. It is separate from high-rate ADC sample transport. Use three 3.3 V MCP23017 I^2C expanders, with address pins strapped for 0x20, 0x21 and 0x22. Start at 100 kbit/s; this is ample for settings and relay sequencing. Fit 2.2 kΩ pullups to 3.3 V on SDA/SCL and verify bus rise time. Decouple every IC with 100 nF. Reset the expanders from the board supervisor and Daisy reset control.

Reserved Daisy pins	Function
D11 / PB8, D12 / PB9	I^2C1 SCL/SDA for configuration
D13 / PB6	Active-low control reset (open-drain plus pullup)
D14 / PB7	ARM; external 100 kΩ pulldown, low means safe
D15 / PC0	Active-low hardware fault interrupt; 3.3 V pullup
D16 / PA3	Heartbeat to independent watchdog
D1–D6	Reserved for SDMMC1, not expander controls
D7–D10	Reserved SPI1 candidate for ADC register configuration

D-names are Seed GPIO indices, not physical header numbers. Confirm these against the actual Seed revision and finish allocation for the frame-sync receiver and future DAC before layout. ADC configuration needs 1.8/3.3 V translation. The data port is not the register SPI bus.

Device	Port bits	Meaning
0x20	GPA[2i], GPA[2i+1]	Channel $i = 0..3$: G2, G11. 00 = gain 1; 10 = 2; 01 = 11; 11 invalid (ordered pair G2,G11).
0x20	GPB[0..3]	AUDIO[i]: 0 wideband, 1 audio. Each bit drives both DPDT capacitor-bank relays for that channel.
0x21	GPA[0..3]	RX_REQ[i], normally-open RX contact request
0x21	GPA[4..7]	TX_REQ[i], normally-open TX contact request
0x21	GPB[0..3]	DAMP_RELEASE[i]; 0 connects a default damping resistor, 1 requests its release
0x22	GPA[0..3]	Power-good, reserved (pull low), TX limit fault, watchdog fault
0x22	GPB[0..3]	PIEZO_QUIET[i], from high-impedance residual-voltage detectors

All unassigned controls stay low. Do not use GPA7/GPB7 as status inputs; check the purchased expander revision's errata. Drive relay coils through MOSFETs with flyback suppression, not directly from expander pins.

Coil driver and capacitor selection. Use this driver for each coil group. Select Q for its guaranteed on-resistance at a 2.5–3.3 V gate drive and the complete coil current. The 100 kΩ gate pulldown keeps it off while expander pins are inputs. Calculate total coil power and measure release time with the flyback network fitted.

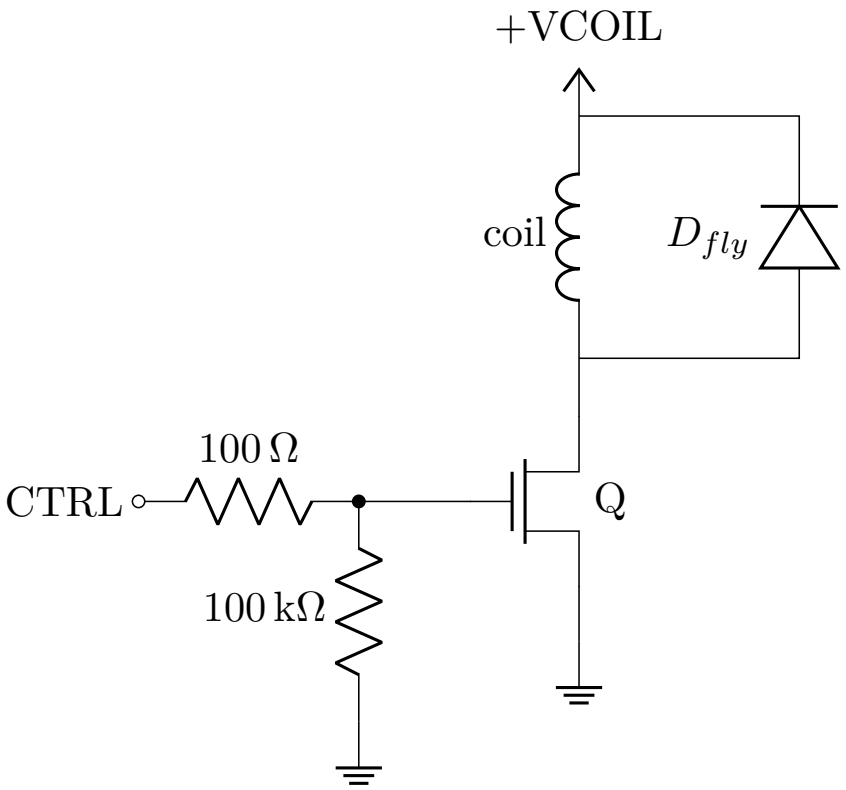


Figure 11: De-energized is the default. Flyback diode cathode faces the coil supply.

Each capacitor in a filter section uses one changeover contact as shown. Two DPDT relays provide the four poles per channel. Contact position during transfer is unspecified: stop accepting samples while switching.

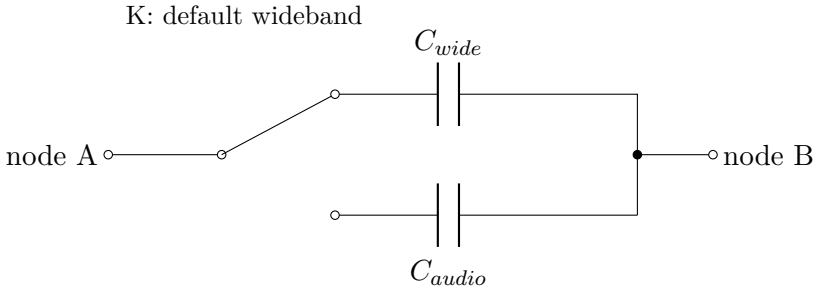


Figure 12: One real capacitor-bank relay pole. Node A/B are the two capacitor terminals in the filter schematic.

Reset and firmware contract.

1. Hardware holds ARM low, TX amplifier shutdown asserted, RX/TX contacts open, damping engaged. Gain defaults to unity and filters to wideband. A watchdog timeout or any fault returns to this state independently of I^2C.

2. Firmware keeps ARM low, writes zero to OLATA/B *before* setting IODIR outputs, sets status

pins as inputs and reads back the configuration. It then programs ADC registers and validates sample framing.

3. The host API stages `gain[4]`, `filter[4]`, `mode[4]`, `txAmplitudeLimit`, `txCurrentLimit`, `txBurstLimit` and `configEpoch`. Reject invalid gain pairs, RX+TX requests, missing calibration, and limits above hardware ceilings. These are firmware fields, not invented ADC registers.

4. A commit blanks acquisition, disables TX, opens both paths, applies the new settings and reads them back. Wait for measured relay bounce and analog settling. The 4.82 Hz high-pass has $\tau = 33$ ms: $7\tau = 231$ ms. Use at least 250 ms blanking after a gain/filter/path change initially, and extend it if measured recovery is slower. Increment the epoch and record the first valid sample index; never present the gap as continuous valid samples.

Gain is held constant during a capture. Auto-ranging may run beforehand; AGC is off by default to preserve amplitude and interchannel comparisons.

Hardware interlock and shared-piezo routing. Use two normally-open contacts per channel: PIEZO–K_RX–input clamps and TX_OUT–K_TX–PIEZO. A separate normally-closed damping contact connects a rated resistor across the piezo in SAFE. Connect both piezo terminals through the routing network if the eventual TX amplifier is bridged; the earlier single-ended ground return is only valid for single-ended drive.

Requests are not coil enables. A hardware sequencer gates them with power-good, ARM, watchdog and fault state. In steady state:

$$RX_{en} = SAFE_{ok}\, RX_{req}\, \overline{TX_{req}}, \qquad TX_{en} = SAFE_{ok}\, TX_{req}\, \overline{RX_{req}}.$$

It must also enforce a both-open interval on every direction change; the Boolean equations alone do not handle relay release delay. Start with a 20 ms minimum dead time and increase it beyond worst-case release, bounce and detector settling. The sequencer/gate and relay part numbers remain to be selected and validated; there is no claim of a built hardware interlock.

RX to TX: blank samples, disable the amplifier, open RX, observe dead time, release damping, close TX and only then enable a ramped waveform. TX to RX: ramp to zero, disable the amplifier, open TX, engage damping, and wait for PIEZO_QUIET before opening damping and closing RX. The residual-voltage threshold and maximum wait come from measured ringdown and the selected gain; a timeout returns SAFE. The diode clamps cannot replace the T/R switch.

The transmit DAC/driver remain an unfinished design, targeting up to 200 kHz waveform output. Until a qualified piezo-specific voltage, current, burst duration and duty-cycle profile exists, the only valid TX amplitude is zero. Calculate capacitive current $I_{pk} = 2\pi f C_P V_{pk}$ and energy $E = \frac{1}{2} C_P V_{pk}^2$; measure resonant impedance and heating as well. Hardware current/voltage limits and amplifier shutdown must work without firmware. Input clamp ratings do not establish the piezo's safe drive rating.

Verification required. Test power-up/down, Daisy reset, stuck I^2C, stopped heartbeat, contradictory requests and input-overload pulses. Check no RX/TX overlap, muted relay transitions, invalid-sample marking, configuration readback and recording metadata. The control map is specified; firmware and board-level timing still need implementation. MCP23017 datasheet and the Seed pinout are the implementation references.

Digital recording path. Figure 13 shows the current digital recording path.

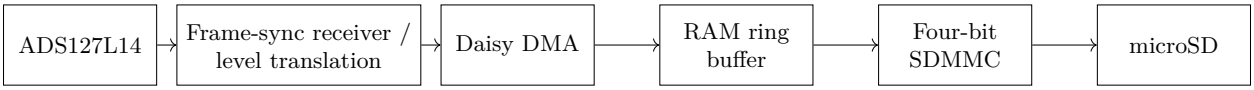


Figure 13: Proposed digital recording path.

The ADS127L14 is the leading prototype candidate, not a finalized selection. The high-impedance voltage preamp is the working front-end choice; a charge-mode amplifier remains an alternative pending sensor measurements. The baseline specifies Daisy-controlled filter banks, a fixed nominal 5 Hz high-pass, and three gain settings; the component-level sheets above implement that starting point. The boxes in Figure 5 are functional blocks: the protection and the driver/filter components must be designed together, and the filtering and gain may be distributed across stages to avoid overload.

Research target. The current prototype target is an electrical passband of 20 Hz–200 kHz at 512 ksample/s per channel, with four simultaneous 24-bit channels. This is not a guaranteed bandwidth for the assembled hydrophone, nor a demonstrated Daisy recording rate. The candidate parts, the rate and storage calculations, the sources and the remaining checks are recorded below.

Protection. Protect both interfaces. Sensor-side protection handles piezo shock and handling transients before they reach the preamp; ADC-side current limiting and clamping handle excessive amplifier output and power sequencing. Clipping above the ADC's full scale is not by itself proof of damage, but exceeding the absolute maximum voltage or current ratings can cause damage. Define the gain, driver supplies, bias and clamps against the selected ADC's ratings, including in powered-off conditions. The leakage and capacitance of the protection must not compromise the sensor's sensitivity or bandwidth.

Cutoff selection. Daisy selects both capacitor banks through the control sheet's relay drivers. The controls are part of the baseline, not a future extension. Change modes only in the blanked configuration state, and record each configuration epoch. The relay contact parasitics must fit the filter tolerance budget.

Preamp choice. Evaluate a high-input-impedance voltage amplifier against a charge amplifier after measuring the disk and cable capacitance. Connecting the sensor directly can load the capacitive sensor, alter its sensitivity and frequency response, provide inadequate gain, and expose the input to mechanical-shock transients. These are risks to characterize, not measured failures of this assembly.

Filtering. Use analog filtering to limit unwanted signals before conversion, coordinated with the codec's internal filtering. Digital filtering can refine the selected band but cannot undo clipping or aliasing. The baseline high-pass is nominally 5 Hz; selectable higher corners can be added if measured motion/thruster noise consumes headroom. Avoid excessive gain ahead of the filtering.

The Daisy's 96 kHz limit. The Daisy documentation specifies up to 96 ksample/s for onboard audio. Its Nyquist frequency is therefore 48 kHz, which is not a guaranteed usable passband. This

motivates the requirement for an external converter; the onboard codec is no longer the bandwidth target. Select the external ADC from the desired acoustic passband, with a sample rate above twice its upper edge and an adequate filter transition margin. Verify the ADC's actual passband and the host's throughput, not only the advertised sample rate.

Mechanical response. The acrylic, adhesive and air-backed disk assembly has its own mechanical response. Measure it underwater: electrical bandwidth alone does not establish acoustic bandwidth.

Questions before completing the spec

- Is phase matching needed for localization? Four channels are confirmed; define the allowable skew and phase mismatch.
- What measured noise floor, sensitivity and maximum input amplitude must the electronics support? The electrical bandwidth and recording targets are specified above; the host feasibility and power budget remain open.
- What are the disk dimensions, model and capacitance, the cable type and capacitance (the length is approximately 2 ft), and the preamp location?
- What are the expected signal amplitudes, the target acoustic frequencies, and the maximum shock and transient conditions?
- Is an additional high-pass preset worth adding after the two Daisy-controlled banks pass their tests?

Validation to define

Measure loading, noise, gain and headroom, electrical frequency response, out-of-band rejection, overload recovery, and stability with representative disk and cable capacitance. Verify every filter mode at its intended sample rate. Measure the underwater acoustic response, and the channel phase consistency if the array is used for localization. Use the acceptance targets above for frequency response and recording; set noise and transient limits after sensor characterization.

References

- Daisy audio documentation — the onboard sample-rate limit.
- Daisy Seed datasheet — verify against the installed hardware revision.
- TI: Signal Conditioning Piezoelectric Sensors — voltage- and charge-mode front ends.
- TI: ADC Input Protection — amplifier/ADC overvoltage and clamping considerations.

Preliminary propagation and recording calculations

Reproduce these numbers with `python3 calculate_capture_budget.py`. The values come directly from the existing snapshot's absorption and noise functions, evaluated without changing the migrating research files. Both `figs/range/acoustic_range.png` and `figs/noise/noise_floor.png` were reviewed.

The conditions match the plots: fresh water at 20 °C, salinity 0, depth 3 m, pH 7.4; and seawater at 20 °C, salinity 35, depth 10 m, pH 8. The ambient noise is the plot's assumed quiet-lake / moderate-ocean model, not measured site noise. The power spectral density (PSD) values in Table 1 are in dB re 1 μPa^2/Hz. The freshwater model omits real-lake sediment, bubbles and scattering, and the noise model is illustrative, especially its lake approximation.

Frequency (kHz)	Fresh absorption / 10 m (dB)	Sea absorption / 10 m (dB)	Lake noise PSD	Ocean noise PSD
40	0.004	0.087	28.7	34.6
100	0.022	0.380	26.6	29.1
200	0.088	0.770	31.5	31.8
500	0.550	1.465	39.3	39.4
750	1.238	2.187	42.9	42.9
1000	2.201	3.162	45.4	45.4

Table 1: Modelled absorption over 10 m and ambient noise PSD (dB re 1 μPa^2/Hz).

Per-channel ksample/s	Packed 24-bit MB/s	32-bit MB/s	Packed GB/hour	Four 32-bit slots: Mbit/s
192	2.304	3.072	8.294	24.576
256	3.072	4.096	11.059	32.768
512	6.144	8.192	22.118	65.536
1000	12.000	16.000	43.200	128.000
2000	24.000	32.000	86.400	256.000

Table 2: Recording data rates for four simultaneous channels.

All storage units in Table 2 are decimal, and the table excludes framing, metadata and filesystem overhead. Rates are per channel, for four simultaneous channels. "24-bit" refers to the stored word length, not the achieved effective resolution. The four-slot serial clock calculation is an interface budget, not proof that the Seed supports those clocks; two stereo lanes would halve the per-lane bit clock if the converter supports them.

What the plots establish. At 200 kHz, the modelled absorption over 10 m is only 0.088 dB in fresh water and 0.770 dB in seawater. Even at 1 MHz, only 2.20 dB (fresh) and 3.16 dB (sea) are lost to absorption. A 10 m range requirement alone therefore does not choose an upper cutoff. The plotted "3 dB range" is an arbitrary absorption criterion, not a detection range. Spherical spreading adds approximately 20 dB of loss from 1 m to 10 m, before multipath, directivity and receiver effects. Use the acoustic curves, not the EM curves, for hydrophones.

At 200 kHz the modelled ambient PSD is about 31.5–31.8 dB re 1 μPa^2/Hz (about 38–39 μPa/$\sqrt{\text{Hz}}$). That is above the model's quieter 50–100 kHz region, but it does not establish whether useful signals exist there. A locally flat 1 kHz analysis band adds 30 dB to the PSD, giving about 61.5–61.8 dB re 1 μPa of band noise; larger bands must integrate the varying PSD. Electronics and vehicle self-noise must be added in linear power, with the measured frequency-dependent hydrophone sensitivity converting electrical noise to equivalent acoustic noise.

For a source level SL specified at 1 m, a first direct-path estimate of the received level is

$$\text{received level} = \text{SL} - 20\log_{10}\left(\frac{r}{1\,\text{m}}\right) - \alpha\,(r - 1\,\text{m}),$$

which is then compared with the total noise in the actual detection bandwidth and the required detection SNR. Neither the source spectrum and level, the hydrophone sensitivity nor the required SNR is known, so a defensible maximum useful frequency or detection range cannot yet be calculated. Active sonar adds a return path and target strength, and needs a different budget.

Seed feasibility and storage. The checked STM32H750 datasheet documents SAI audio sampling up to 192 ksample/s, and current libDaisy exposes a 192 kHz setting. This is separate from the onboard codec's 96 ksample/s limit. The Seed exposes SAI2 and SDMMC1 on distinct header pins; pin availability is promising, but four-channel operation still needs clock/framing and DMA validation. Do not assume a 512 ksample/s audio path is supported. Higher-rate acquisition may require a suitable SPI ADC and custom DMA firmware, or a different controller; verify the guaranteed interface timing on the actual MCU revision.

A 200 kHz passband requires more than 400 ksample/s per channel. The 512 ksample/s baseline leaves 56 kHz between the 200 kHz passband edge and Nyquist. This is a design target, not a demonstrated host rate. At four channels it requires 6.144 MB/s of packed storage, or 8.192 MB/s with 32-bit words; packed recording uses 22.118 GB/hour. A 192 ksample/s path has a 96 kHz Nyquist ceiling and cannot capture 200 kHz.

Use a native four-bit SDMMC microSD interface, not QSPI. At an assumed supported SD clock of 25 MHz, its raw ceiling is 12.5 MB/s; at 50 MHz it is 25 MB/s, before all overhead and card busy periods. These are calculated bus ceilings, not measured sustained writes or established board clock settings. For the baseline 512 ksample/s case, provision and measure at least 2× average write-rate headroom (12.288 MB/s packed), with independent DMA acquisition and a ring buffer.

The buffer requirement is the incoming bytes per second multiplied by the maximum storage stall, plus margin and the buffers already in flight. A hypothetical 0.5 s stall needs 3.072 MB packed, or 4.096 MB in 32-bit format, at 512 ksample/s. A 16 MiB ring holds about 2.73 s packed or 2.05 s in 32-bit format; this is capacity, not a guarantee against all stalls. The Seed's documented 64 MB SDRAM could provide the buffering, subject to DMA access, cache coherency, bandwidth and other allocations. Card latency must be measured under representative fill, filesystem and operating conditions. Log dropped frames and overruns explicitly, preallocate files, and plan file segmentation (FAT32 has a 4 GiB file limit).

Decisions still pending. Do not select 200 kHz solely from the range curves. First measure the assembled hydrophones' response and noise, and identify the source spectra of interest. Compare a documented ≤192 ksample/s external acquisition path with a higher-rate research path using Table 2. Preserve selectable analog cutoff modes matched to the selected ADC and rate. Before layout, finish the gain/noise/anti-alias budget, the converter and host timing, channel skew, storage endurance and capacity, the maximum card stall, the supply and current budget, and the pin allocation. No hardware throughput or acoustic calibration has been tested as part of this draft.

Additional primary references.

- STM32H750 datasheet, SAI and SDMMC.
- libDaisy SAI configuration.
- libDaisy SDMMC implementation.

- NPL absorption model background and limits.

Component shortlist and achievable-rate distinctions

The parts below are discussion candidates, not a validated BOM or a measured maximum Seed recording rate.

- **Documented SAI path.** A four-channel PCM1840 ADC at 192 ksample/s with a custom four-slot TDM receive configuration. Four 32-bit slots require a 24.576 MHz BCLK, and the firmware/DMA integration must be verified. Its linear-phase filter at 192 ksample/s specifies ± 0.05 dB only through $0.3 \times F_s = 57.6$ kHz (Table 14), so do not promise an 80–90 kHz flat passband from the sample-rate label. Disable dynamic gain behaviour for calibrated array measurements. (PCM1840 datasheet.)
- **Higher-bandwidth prototype candidate.** The ADS127L14: four simultaneous 24-bit channels in a 512 ksample/s wideband mode. Its Table 7-2 gives a -0.1 dB edge of 211.2 kHz and a -3 dB edge of 223.9 kHz. It needs a reference, a clock, analog supplies, and 1.8 V I/O translation to the Seed. Its faster sinc modes are a different bandwidth/aliasing trade-off, not the same wideband specification. (ADS127L14 datasheet.)
- **ADS127L14 data interface—correction.** SPI programs registers; conversion data leave the separate FSYNC/DCLK/DOUT frame-sync port (datasheet Section 7.4.10). A faster SPI register read is not an acquisition solution. Payload is 49.152 Mbit/s; status/CRC increase the wire budget. Table 7-15 shows 65.536 MHz DCLK for TDM ratio 4, 24-bit packets at 512 ksample/s, generated from 65.536 MHz CLKIN divided by two for the ADC core. ADC variant, active lanes and packet layout must be checked explicitly. Evaluate a synchronized serial receiver or bridge/FIFO; a bridge part is not selected. Verify frame alignment, voltage translation, timing and DMA under SD load before committing to the Seed.
- **Front-end candidate.** Eight OPA1656 dual FET-input packages for the drawn four-channel implementation: one buffer, one gain stage and two filter stages per channel. Verify capacitance, noise, stability, input common-mode and supply requirements. (OPA1656.)
- **ADS-path driver candidate.** One THS4551 differential driver per channel (the part used in the ADC datasheet's application example). The board also needs selectable RC filters and signal relays, input and ADC protection, a precision reference and buffer, an oscillator, low-noise regulators, and suitable fixed-direction level translators. Part numbers for those remain pending the electrical budget.
- **Initial card candidate.** A SanDisk MAX ENDURANCE 256 GB, U3/V30 card, via four-bit SDMMC. The vendor rates it at up to 40 MB/s writing; that is not a guaranteed Seed write speed or latency bound. At 6.144 MB/s packed, a nominal 256 GB card holds 11.57 hours before capacity, metadata and reserve overhead; a 128 GB card holds 5.79 hours. Benchmark the exact card and firmware, and allocate RAM for stalls. Confirm SDXC filesystem support, or use compatible formatting and file segmentation. (Manufacturer card data sheet.)

Working recommendation. Evaluate the ADS127L14 capture-and-record path for approximately 200 kHz of research bandwidth, and retain the PCM1840 option only if its narrower bandwidth meets the experiment. The highest reliable end-to-end Seed rate remains unmeasured.